BPF加载器与签名方案详解

use_loader 机制与主流签名方案技术解析

The 4th eBPF Developer Conference (eBPFDC 2026)

天翼云科技 牛万里

陕西 · 西安

2026年4月18日

01 技术背景

技术背景

行业安全合规需求

行业对**BPF 程序可信加载**提出明确需求，亟待补齐基础设施安全短板。

核心安全目标

确保内核中运行的 BPF 程序**来源合法、未被篡改**，从源头防止恶意注入。

构建可信执行环境

核心诉求：打造一个**可验证、防篡改、可追溯**的安全运行时环境。

核心技术手段

引入**数字签名机制**是实现 BPF 环境全生命周期可信验证的最佳实践方案。

02 eBPF签名核心难题

eBPF 签名核心难题解析

核心痛点：BPF 加载的运行时动态变换

- CO-RE 重定位：指令需根据内核版本在加载阶段动态修正，非编译期固定值。
- Map 重定位：Map 的文件描述符(FD)需在加载时动态绑定，无法提前固化。

直接后果：传统静态签名机制完全失效

由于上述动态变换，内核最终执行的指令 ≠ 编译生成的原始 .o 文件，依赖二进制指纹的传统方案无法匹配。

解决方案与潜在攻击风险

- 应对策略：加载时传递 BTF 信息由校验器完成 CO-RE；Map 重定位使用 BPF_PSEUDO_MAP_IDX。
- 安全隐患：MAP 创建与 BPF 加载是独立系统调用，中间存在攻击窗口，易遭“替换 MAP fd”注入攻击。

03 bpftool -S 签名

基于 use_loader 加载器的解决方案

加载器模式核心设计（Loader Mode）

参数定义：-L / --use-loader

-L, --use-loader

Load program as a "loader" program. This is useful to debug the generation of such programs. When this option is in use, bpftool attempts to load the programs from the object file into the kernel, but does not pin them (therefore, the PATH must not be provided).

核心架构与逻辑

用户态：仅加载内核态 BPF 加载器

业务程序：不再由用户态直接加载

内核态：加载器独立完成校验 → 加载全流程

工程实践：生成 Skel Header

提取 .bpf.o 内容到 .skel.h

```
bpftool gen skeleton -L \
trace.bpf.o > trace.bpf.skel.h
```

加载器工作流程

编译阶段

生成内核态加载器
用户态通过bpftool生成内核态BPF加载器程序，作为执行入口。

创建加载器数据MAP结构
填充BPF指令、BTF信息及各阶段运行参数。

执行阶段

内核态加载业务程序
加载器执行加载任务，创建业务MAP，并载入业务BPF字节码。

内核校验与CO-RE重定位
内核校验器自动完成CO-RE重定位，确保程序在不同内核版本兼容。

加载器 MAP 重定位器工作流程

重定位策略优化

当启用 gen_loader 时，使用**BPF_PSEUDO_MAP_IDX**模式，指令中直接存储 Map 数组下标；运行时通过下标映射到真实 FD，避免了传统方案中动态修改指令。

```
bpftool-comment/libbpf/src/libbpf.c: 6346

for (i = 0; i < prog->nr_reloc; i++) {
    struct reloc_desc *relo = &prog->reloc_desc[i];
    struct bpf_insn *insn = &prog->insns[relo->insn_idx];

    switch (relo->type) {
    case RELO_LD64:
        map = &obj->maps[relo->map_idx];
        if (obj->gen_loader) {
            insn[0].src_reg = BPF_PSEUDO_MAP_IDX;
            insn[0].imm = relo->map_idx;
        } else if (map->autocreate) {
            insn[0].src_reg = BPF_PSEUDO_MAP_FD;
            insn[0].imm = map->fd;
```

加载器 skel.h 关键组成与内存布局

核心数据结构定义（opts_data / opts_insn）

opts_data：存储业务 BPF 指令、BTF 信息、各阶段加载参数（如fd_array、map_attr等），是业务程序的“容器”。

opts_insn：加载器自身的 eBPF 指令集，由 bpftool 在编译阶段自动生成，负责实际的加载与重定位逻辑。

```
-static const char data[] __attribute__((__aligned__(8))) = "\
+static const char opts_data[] __attribute__((__aligned__(8))) = "\
+static const char opts_insn[] __attribute__((__aligned__(8))) = "\
```

内存布局与核心加载流程（Offset Map）

0 ~ 1280（bpf_gen__init）：预留 fd_array 空间，存储后续 map/prog 的 fd 返回值。

1280 ~ 31356（bpf_gen__load_btf）：写入 .BTF 原始数据与加载属性，完成 CO-RE 重定位基础。

31360+（bpf_gen__prog_load）：写入 license、eBPF 指令流、func_info/line_info 及重定位表，最终加载程序。

关键参考：readelf 查看 .o 文件节区

通过 `readelf -S` 可查看 .text(指令)，.BTF，.BTF.ext，.maps(map定义) 等节区的偏移与大小，这正是 opts_data 中需要序列化并加载的核心内容。

加载器执行系统调用流程

```
strace -e bpf ./trace
bpf(BPF_MAP_CREATE, {map_type=ARRAY, key_sz=4, value_sz=32392, max_entries=1, ...}, 92) = 3
bpf(BPF_MAP_UPDATE_ELEM, {map_fd=3, key=0x7ffc..., value=0x404100, flags=ANY}, 32) = 0


bpf(BPF_PROG_LOAD, {prog_type=SYSCALL, insn_cnt=158, ..., prog_name="__loader.prog"}, 168) = 4
bpf(BPF_PROG_TEST_RUN, {test={prog_fd=4, ctx_in=0x2071ceb0, ...}}, 80) = 0
```

BPF_MAP_CREATE/UPDATE 阶段加载基础信息，BPF_PROG_LOAD 加载加载器本体。在**BPF_PROG_TEST_RUN**执行时，加载器在用户态通过一次系统调用，完成业务 Map 初始化与业务 BPF 程序的完整加载。

内核层：签名合法性

当加载器程序被内核加载时，优先校验其数字签名。确保加载器本身来源可信，未被恶意篡改。

加载器层：哈希完整性

加载器加载业务BPF程序前，计算并比对程序哈希值。确保业务逻辑代码完整，无恶意指令注入。

双重验证：执行放行

仅当内核层签名验证 AND 加载器层哈希验证全部通过，业务BPF程序才被允许在内核态执行。

bpftool -S skel.h 关键组成（技术变化）

opts_sig：加载器指令签名
加载器 LOAD 时传入，用于内核对指令进行第一次合法性校验。

opts_data：业务 BPF 核心数据
包含 BPF 镜像、BTF 信息、原始指令集及重定位相关元数据。

opts_insn：加载器 eBPF 指令
在指令中添加 opts_data 的 Hash 值，执行时进行二次完整性校验。

skel.h

```
static const char opts_sig[] __attribute__((aligned(8))) = \
"........ opts_sig ........";
static const char opts_excl_hash[] __attribute__((aligned(8))) = \
"........ opts_excl_hash ........";
// 将签名信息注入加载器选项
opts.signature = (void *)opts_sig;
opts.signature_sz = sizeof(opts_sig) - 1;
```

核心校验逻辑（emit_signature_match）

校验入口：核心逻辑位于加载器函数 emit_signature_match 中

内核侧取值：从内核 MAP 结构中读取内核通过 bpf_obj_get_info_by_fd 计算出的哈希值。

用户态预置：加载器内置编译期预计算好的业务程序哈希，作为指令中的立即数参与比对。

强校验退出：逐双字比对，一旦不匹配则立即跳转到 cleanup 清理资源并退出。

关键代码片段（C Language）

```c
static void emit_signature_match(struct bpf_gen *gen) {
    inti;// 循环比对SHA256的每一个双字
    for(i = 0; i < SHA256_DWORD_SIZE; i++) {
        emit2(gen, BPF_LD_IMM64_RAW(...));// R1: 内核计算的Map Hash
        emit(gen, BPF_LDX_MEM(...));// R2: 加载内核Hash值
        emit2(gen, BPF_LD_IMM64_RAW(...));// R3: 加载用户态预置Hash
        ......
        emit(gen, BPF_JMP_REG(BPF_JNE, R2, R3, off));// 不匹配则跳转退出
    }
}
```

bpftool -S 用户态生成流程（技术变化）

01

计算哈希并嵌入加载器指令

核心技术变化点：将业务BPF程序的哈希值植入加载器，建立完整性校验基准。

02

使用证书对加载器进行签名

安全关键步骤：利用私钥对加载器二进制进行数字签名，证明其来源的合法性与未篡改性。

03

生成带签名信息的 Skeleton 头文件

最终交付产物：将签名块与程序元数据封装，输出为用户态程序可直接包含的 skel 头文件。

bpftool -S 内核态验证流程（技术变化）

内核校验签名

LOAD 加载器时，将加载器签名传递给内核，由内核验证签名的合法性，确保加载器来源可信。

校验程序哈希

加载器启动阶段，校验业务程序的哈希值。该哈希由用户态提前生成，并已嵌入到加载器的指令立即数中。

正式加载程序

当加载器签名与业务程序哈希双重校验均通过后，校验流程完成，加载用户程序。

bpftool -S 核心技术特性

签名对象为加载器

签名对象锁定为加载器自身，**完全不受业务程序动态重定位影响**，保证校验基准的一致性。

原生兼容 CO-RE 机制

完美兼容 BPF CO-RE (Compile Once - Run Everywhere) 标准，同时支持**MAP 动态重定位**功能。

内核态原子性安全校验

校验逻辑全程在内核态执行，**单系统调用完成加载**，用户态无法进行任何绕过与恶意篡改操作。

主线内核原生支持

功能已合入 Linux 主线内核，无任何额外组件依赖，降低生产环境的部署与维护成本。

04 Hornet签名方案

Hornet 签名模式实现

01. 签名生成指令（CLI）

```
scripts/hornet/gen_sig \
-data insn \
-cert signer.crt \
-key signer.key \
--add-hash data \
-out sign
```

02. 核心逻辑（gen_sig.c）

```
// 计算并附加业务数据 Hash
for (i = 0; i < hash_count; i++) {
sha256(hashes[i].file, buf, &len);
add_hash(set, buf, len, i);
}
// 将 Hash 序列作为扩展属性加入签名
CMS_signed_add1_attr_by_OBJ(si, oid, ...);
```

03. 签名注入（.skel.h）

```
+static const char opts_sig[] = "...sign...";

+opts.signature = (void *)opts_sig;
+opts.signature_sz = sizeof(opts_sig) - 1;
```

Hornet 验签流程（与 bpftool -S 对比）

触发 LSM 钩子

当调用 `BPF_PROG_LOAD` 加载 eBPF 程序时，内核自动触发 LSM 安全钩子进行拦截。

多重安全校验

LSM 模块统一完成对 eBPF 程序的数字证书、代码签名及哈希完整性的全面校验。

放行加载执行

只有当所有校验项均通过时，内核才会允许加载器将 eBPF 程序加载并运行。

核心差异对比（vs bpftool -S）

在传统的 `bpftool -S` 方案中，哈希校验逻辑是在加载器内部实现的；而 Hornet 架构下，哈希校验完全由内核 LSM 模块接管执行。

Hornet 验签核心流程与机制

校验位置：LSM 钩子中
脱离加载器指令内部，由 LSM 安全模块统一接管

读取来源：attr->fd_array
直接取出加载器传入的 DATA map fd，获取业务数据

内核计算：map_get_hash
内核态直接计算 MAP 内存数据的 SHA256 哈希值

比对逻辑：直接对比
与签名扩展属性(xattr)中预计算的哈希值进行 memcmp

核心差异：加载器内部不再执行哈希校验，完全由 LSM 执行

```
// 关键实现：hornet_lsm.c

static int hornet_verify_hashes(struct hornet_maps *maps, ctx) {
// 1. 从 attr->fd_array 读取加载器传入的 Map FD
copy_from_bpfptr_offset(&map_fd, maps->fd_array, idx * sizeof(u32));
// 2. 获取内核态 Map 结构体指针
struct bpf_map *map = fd_file(f)->private_data;
// 3. 内核计算 Hash (SHA256)
map->ops->map_get_hash(map, SHA256_SIZE, calc_hash);
// 4. 与签名中的预计算 Hash 进行比对
if (!memcmp(calc_hash, ctx->pre_hashes, SHA256_SIZE)) return BADSIG;
}
```

两种方案技术特性对比

bpftool -S 方案（已合入主线）

主线状态	技术成熟，已正式合入 Linux 内核主线
技术基座	完全基于原生 bpftool-L 加载器构建
Hash 载体	业务 Hash 信息嵌入在加载器指令的立即数中
验证	内核加载时签名校验 + 加载器运行时 Hash 逻辑验证

Hornet LSM 方案（社区提案阶段）

主线状态	处于社区讨论与提案阶段，尚未合入主线
技术基座	基于 bpftool-L + Linux 安全模块(LSM)机制
Hash 载体	业务 Hash 存储在签名文件的扩展属性段中
验证	内核加载时签名校验 + LSM 钩子加载时拦截验证

谢谢聆听